

README

🏥 Hospital Command Center & Resource Optimization

A full-stack **Hospital Command Center** web application that helps hospital administrators monitor and manage beds, doctors, nurses, equipment, and patients in real time, and automatically generates resource alerts and recommendations.

Enter the hospital's current condition → backend processes the data → calculates availability & utilization → identifies shortages → dashboard updates instantly.

☆☆ What It Does

- 🛏 Tracks bed occupancy (ICU, General, Emergency)
- 👨‍⚕ Tracks doctor and 👩‍⚕ nurse availability by department
- 🩺 Monitors medical equipment (ventilators, ECG, patient monitors)
- 👤 📊 Tracks patient counts
- 📢 Generates automatic resource alerts (NORMAL / WARNING / CRITICAL)
- 📊 Visualizes resource utilization with Chart.js
- 🔄 Supports **Normal Condition** and **Emergency Surge** simulations for demos

Note: Data in this prototype is real-time, user-entered operational data (not live hospital sensors or a hospital database). When staff enter values, they are sent to the backend and the dashboard refreshes immediately.

🛠 Tech Stack

Layer	Technology
Frontend	HTML5, CSS3, JavaScript
Visualization	Chart.js
Backend	Python 3.13, FastAPI
API Server	Uvicorn
Data Validation	Pydantic
Communication	REST API + JSON
Cross-Origin Communication	CORS
API Testing	Swagger / OpenAPI
IDE	Visual Studio Code
Data Storage	Python in-memory data structure (current prototype)

See [ARCHITECTURE.md](#) for the full system design and [FEATURES.md](#) for implementation details.

📁 Project Structure

```
├── index.html      # Main dashboard UI
├── login.html      # Login page (demo credentials)
├── main.py         # FastAPI backend & API routes
├── data.py         # In-memory hospital data
├── README.md
├── PROBLEM_STATEMENT.md
├── ARCHITECTURE.md
├── FEATURES.md
└── DEVELOPMENT_HISTORY.md
```

🔧 Getting Started

1. Install dependencies

```
pip install fastapi uvicorn pydantic
```

2. Run the backend

```
py -m uvicorn main:app --reload
```

The API will be available at: <http://127.0.0.1:8000>

3. Open the frontend

Open `login.html` (or serve it with Live Server, e.g. <http://127.0.0.1:5500>) and log in to reach `index.html`.

4. Explore the API

Visit <http://127.0.0.1:8000/docs> for interactive Swagger/OpenAPI documentation.

🔗 API Endpoints

Endpoint	Method	Purpose
/status	GET	Get current hospital status
/recommendations	GET	Get alerts / recommendations
/simulate/normal	POST	Reset to normal hospital condition
/simulate/surge	POST	Simulate an emergency surge
/update	POST	Submit actual current hospital data

⚠️ Current Limitations

- Data is stored **in memory** — restarting the FastAPI server resets values back to the defaults in `data.py`.
 - Login uses **hardcoded demo credentials** — this demonstrates role/access control but is **not production-grade authentication**.
 - Optimization is **rule-based**, not machine learning (see [FEATURES.md](#) for details and future ML extension ideas).
-

Documentation

- [PROBLEM_STATEMENT.md](#) — the problem this project solves and how it aligns with the goal
- [ARCHITECTURE.md](#) — system architecture and data flow
- [FEATURES.md](#) — core implementation details (calculations, recommendation engine, simulations)
- [DEVELOPMENT_HISTORY.md](#) — development progress and roadmap

One-Minute Pitch

“Our project is a web-based Hospital Command Center and Resource Optimization system. The frontend is developed using HTML, CSS and JavaScript, with Chart.js for visualization, while the backend is developed using Python and FastAPI. Hospital staff can enter the current number of beds, doctors, nurses, equipment and patients through the web interface. This data is converted into JSON and sent to the FastAPI backend through REST APIs. The backend calculates available resources, occupancy percentages and utilization, and applies rule-based conditions to identify warnings and critical shortages. The processed data is then returned to the frontend, where JavaScript immediately updates the dashboard, status badges, alerts and charts. We also provide normal-condition and emergency-surge simulations to demonstrate how the command center responds to changing hospital demand.”